

Blue Sand

Design Documentation

Version 2.1

August 2024

Table of Contents

Design Overview	4
Counter Unit	5
Enabling and disabling the counter block	5
Changing the Number of cycles for the counter block	6
Counter block modes	6
Change the count direction.	8
Loading main load value	8
Counter programming flow	8
Pulse Width Modulation Block	10
Pulse Width Modulation (PWM)	10
Sinusoidal Pulse Width Modulation (SPWM)	11
Configuring the Modulation block	13
Adjusting the frequency of the PWM signal	14
Adjusting the duty cycle of the PWM signal	14
Adjusting the frequency of the SPWM signal	14
Configuring the Fixed Range	15
Configuring the Phase shift	16
Multi level arbitration Block	17
Weighted Round Robin block	18
Regions	18
Choosing a source	19
Random block	19
Weighted Random block	20
Clock Muxing Unit	20

Clock muxing configuration	21
Fatal error	21
Signals description	22
APB Interface	22
Counter Block Interface	22
Pulse Width Modulation block signals	22
Multi level arbitration block	22
Weighted round robin block	23
Rand arbitration block	24
Weighted random block	24

List of Figures

Figure 1 : block diagram of the Blue-Sand designFunctional description	4
Figure 2 : the basic flow of the Counter	9
Figure 3 : The modulation block	10
Figure 4 : block diagram of the Blue-Sand design	17
Figure 5 : block diagram of the weighted round robin design	18
Figure 6 : Weights distribution	18
Figure 7 : block diagram of the random design	19
Figure 8 : block diagram of the weighted random design	20
Figure 9 block diagram of clock muxing unit	20

List of Tables

Table 1 : the single wrap point change direction actions	6
Table 3 Weights values	19
Table 4 Clock muxing config	21
Table 5 : counter output signals details	22
Table 6 : Modulation output signals details	22
Table 8 Multi level arbitration block signals	23
Table 9 Weighted round robin block	23
Table 10 Rand arbitration block	24
Table 11 Weighted random block	25

Design Overview

Blue sand is an APB-controlled design. This design uses the Advanced peripheral bus (APB) protocol for accessing the internal programmable register control unit. Blue Sand includes several programmable units: The **counter** unit generates a count output that can operate in various modes, the **pulse-modulation** unit generates pulse-width modulation (PWM) and sinusoidal PWM (SPWM) signals with various features, **Multi level arbitration** unit consisting of three arbitration levels, **Clk Muxing** unit controls the operating frequency of the other blocks. All the units are controlled by an internal register control unit. Figure 1 shows the block diagram and the input/output signals of the full design.

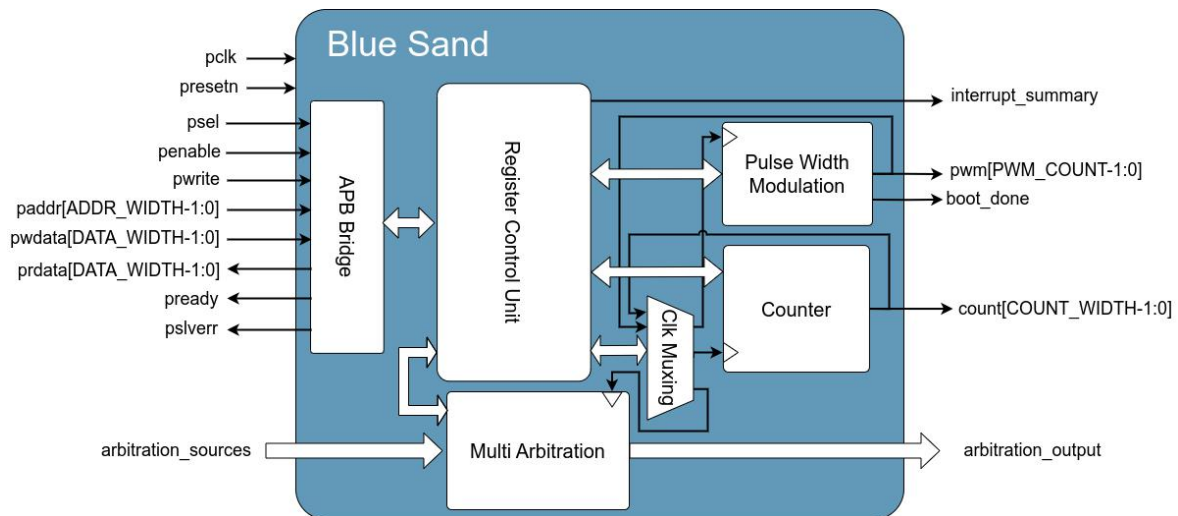


Figure 1: block diagram of the Blue-Sand design

Functional description

The design consists of 6 blocks as described below:

- The APB bridge, reads read and write requests from APB master and transform it to a simple inner protocol and passes it to the register control unit that gives the appropriate response which is transformed back to the APB protocol by the APB bridge.
- The register control block, includes the internal registers that control and monitor the design behavior.
- The counter block, generates the count of width of COUNT_WIDTH which is configured to be 8.
- The pulse width modulation block, generates a PWM or a SPWM signal on PWM_COUNT ports.
- The multi arbitration block, takes requests from 16 sources and arbitrate the requests then outputs it.
- The clk muxing block, generates an operating clock for the pwm block, counter block, and multi arbitration block according to the value of counter output, pwm output, and the configuration in the register control.

COUNTER UNIT

The counter unit is an N-bit counter that is controlled by the register control unit. The counter can count up or down at the rising edge of the clock according to the specified count mode. The counter width can be configured using COUNT_WIDTH parameter (Default value = 8).

Enabling and disabling the counter block

1. Enabling the counter:

- The counter block is enabled by writing 1 on bit [0] of the counter_en_config_reg register.
- When the counter is enabled, it counts up or down according to the configured direction and mode.

2. Disabling the counter

- The counter is disabled by writing 0 on bit [0] of the counter_en_config_reg register.
- When the counter is disabled, it will stop on the last value that was reached.

Changing the Number of cycles for the counter block

The counter_number_of_cycles_config_reg register controls the needed rising edges of the clock to get to the next count value. The register value is mapped to $[1:2^{COUNT_WIDTH}]$. The register value can be only changed if the counter is disabled.

Counter block modes

The mode bits [1:0] of the counter_mode_control_config_reg register controls the mode of the counter block. There are four modes for the counter block:

1. Free running, bits [1:0] = 2'b00
2. Single wrap point, bits [1:0] = 2'b01
3. Double wrap, bits [1:0] = 2'b10
4. Special single wrap point, bits [1:0] = 2'b11

1-Free running mode

- The count will wrap to '0' if it reached the maximum value ($2^{DATA_WIDTH-1}$).
- The count will wrap to the maximum value ($2^{DATA_WIDTH-1}$) if reached '0'.
- Region: 0 - $2^{DATA_WIDTH-1}$.

2-Single wrap point mode

- The count will wrap to the main load value if reached the maximum value ' $2^{DATA_WIDTH-1}$ '.
- The count will wrap to the main load value if reached '0'.
- The count will wrap to '0' if reached the main load value when counting up.
- The count will wrap to the maximum value ' $2^{DATA_WIDTH-1}$ ' if reached the main load value when counting down.
- Regions:
 - (0-Main load)
 - (Main load- $2^{DATA_WIDTH-1}$)
- Changing the counting region:

A switch between the 2 regions happens when the counter mode (up/down direction) is changed so that the counting direction points to a new region while you are at the wrap point (Main load). The following table clarify the switching operation:

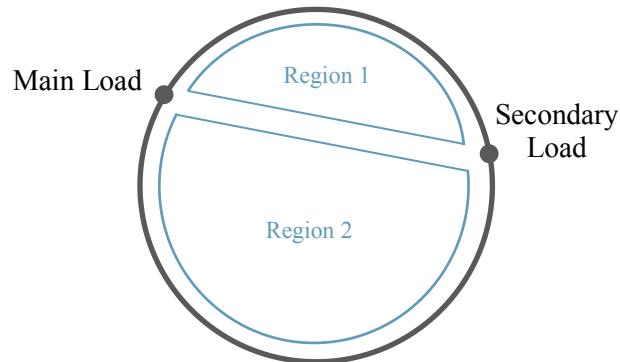
Table 1: the single wrap point change direction actions

Current Region	Count Mode	Action to switch to the other region
0 – Main Load	Up	Cannot be switched directly

	Down	Changing the mode to 'Up' at the (Main load) value
Main load – $2^{\text{DATA_WIDTH}-1}$	Up	Changing the mode to 'Down' at the (Main load) value
	Down	Cannot be switched directly

3- Double wrap point mode

- The count will wrap to the main load value if it reaches the secondary load value.
- The count will wrap to the secondary load value if it reaches the main load value.
- Regions:
 - Inner Region: between Secondary Load and Main Load values.
 - Outer Region: Outside the inner region



- A switch between the 2 regions happens when:
 1. The counter mode (up/down direction) is changed so that the counting direction points to a new region while you are at the wrap points (Main and Secondary points).
 2. If the Secondary Load value is changed so that the current count value is out of the assumed region.

4- special single wrap point mode

- The counter will wrap to the main load value if it reached the maximum value ' $2^{\text{DATA_WIDTH}-1}$ '.
- The counter will wrap to the main load value if reached '0'.
- Region: $0 - 2^{\text{DATA_WIDTH}-1}$

Change the count direction.

The counter direction can be controlled by writing on counter_direction (bit [2]) of the counter_mode_control_config_reg register.

- Counting UP: by writing 1 to bit [2] of the counter_mode_control_config_reg register.
- Counting DOWN: by writing 0 to bit [2] of the counter_mode_control_config_reg register.

Loading main load value

The counter value changes by writing the new value on the counter_main_load_config_reg register.

Counter programming flow

The procedure illustrated in Fig.2 is the initial programming flow of the Counter block.

1. Disable the counter.
2. Load the (Number of cycles).
3. Program the counter modes.
4. Load the main value.
5. If the counter mode is set to (double wrap point), load the second wrap point value by writing on the counter_secondary_load_config_reg register.
6. Enable the counter.

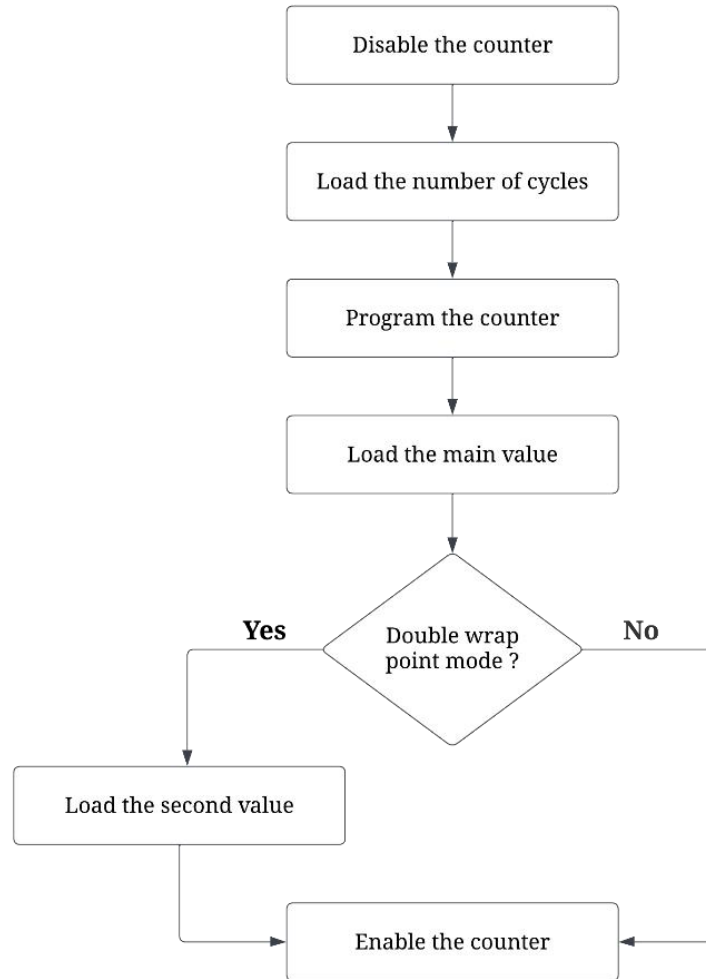


Figure 2: the basic flow of the Counter

PULSE WIDTH MODULATION BLOCK

Pulse width modulation block produces pulse width modulation (PWM) and sinusoidal pulse width modulation (SPWM) signals. A signal is said to be pulse width modulated if the size of the pulses in its output can be adjusted.

The block contains PWM_COUNT generation units each can generate independent pulse width modulation (PWM and SPWM) signals. The following figure shows a general view of the pulse width modulation block.

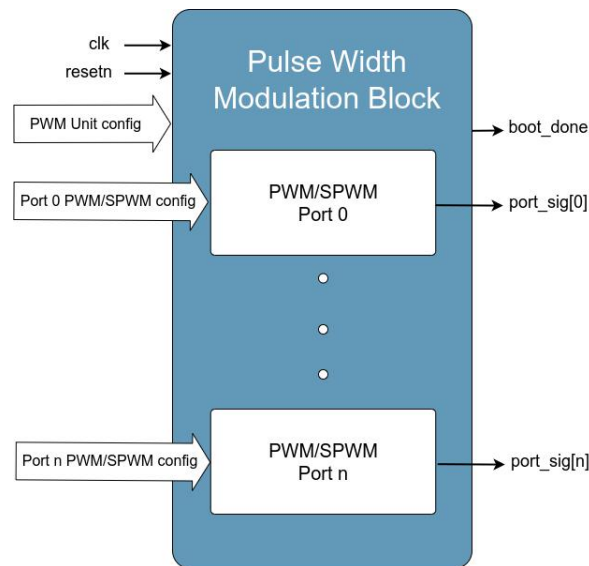
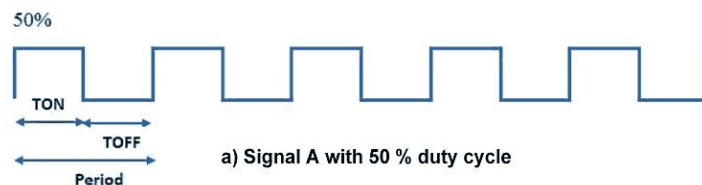


Figure 3: The modulation block

Pulse Width Modulation (PWM)

Pulse Width Modulation (PWM) is a DC voltage controlling method that depends on controlling the average value of a train of pulses signal. The two main characteristics of PWM signal are the **frequency** and the **duty cycle**. The frequency determines the period of the pulse, while the duty cycle can be defined as the percentage of the ON period from the whole period. It controls the average power of the output signal, the higher it is the higher the average power is.



PWM signal is generated according to the PWM port frequency and its duty cycle. they are used for calculating the ON and OFF time periods of the PWM signal. The frequency of the

PWM signal depends on the clock frequency, frequency resolution, and the prescaler using the following equation:

$$PWM_{freq} = \left(\frac{1}{Freq_{resolution} \cdot 2^{prescaler}} \right) Clock_{freq}$$

Calculating the ON and OFF times of the PWM period is done as follows:

ON time:

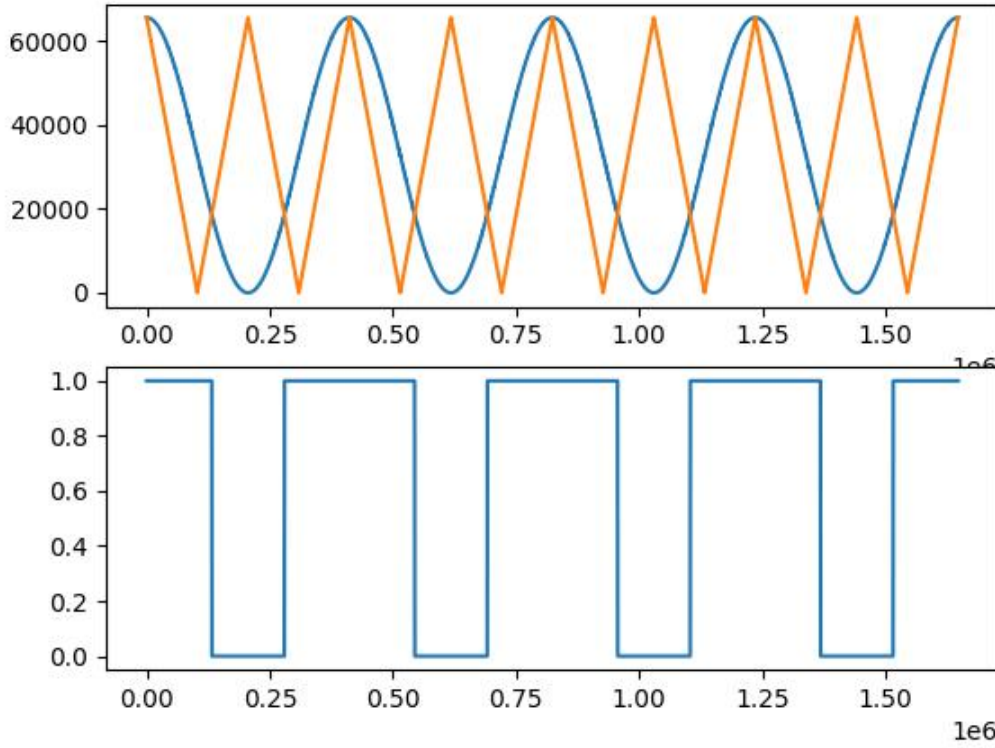
$$T_{on} = \left\lfloor \frac{Duty_Cycle \cdot PWM_{period}}{256} \right\rfloor$$

OFF time:

$$T_{off} = (PWM_{period} - T_{on})$$

Sinusoidal Pulse Width Modulation (SPWM)

Sinusoidal Pulse Width Modulation (SPWM) is another pulse modulation technique. In this modulation, pulses are produced by comparing a reference signal with a carrier signal thus the SPWM signal are ON if and only if the carrier signal is strictly less than the generated sinusoidal. The general behavior of the pulses in the SPWM is that the pulses close to the ends of the half cycle are narrower than the pulses close to the center of the half cycle such that the pulse widths are comparative to the amplitude of a sine wave at that part of the cycle. To change the efficient output voltage, the widths of all pulses are amplified or reduced while keeping the sinusoidal proportionality. The main parameters of SPWM signal are the frequency of the reference (Sinusoidal) and the carrier (triangle) waves. The carrier wave frequency has to be higher than that for the reference wave in order to generate a sinusoidal modulation process.



SPWM generation

Spwm $S(x)$ function is generated according to the following equation.

$$S(x) = \cos(x) \geq \text{trng}(x) \{0 \leq x \leq \frac{\pi}{2} 2^{16}, x: \text{int}\} \{S(x) \in \{0,1\}\} \{\frac{\pi}{2} 2^{16} \approx 102944\}$$

A 3rd order Cosine power series were used to generate the first quarter of the sinusoidal signal:

$$\cos(x) \cong 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} \{0 \leq x \leq \frac{\pi}{2}, x: \text{float}\} \{-1 \leq \cos(x) \leq 1\}$$

The equation were mapped to integers. The following enhancement has been added to the equation of the first quarter:

$$f(x) \cong \frac{\{2^{16} \cdot [\cos(\frac{x}{2^{16}}) + 1]\}}{2} \{0 \leq x \leq \frac{\pi}{2} 2^{16}, x: \text{int}\} \{0 \leq f(x) \leq 2^{16}\}$$

$$f(x) \cong \left\{ 2^{16} - \left\lfloor \frac{x^2}{2^{18}} \right\rfloor + \left\lfloor \frac{\frac{x^4}{2^{52}}}{3} \right\rfloor - \left\lfloor \frac{\frac{x^6}{2^{85}}}{3^2 \cdot 5} \right\rfloor \right\} \{0 \leq x \leq \frac{\pi}{2} 2^{16}, x: int\} \{0 \leq f(x) \leq 2^{16}\}$$

This enhancement will make the sinusoidal wave applicable on integers without the need of float calculations.

The remaining three quarters are generated by shifting and rotating the first quarter values. This is done to reduce the mathematical operation and memory size needed to create the full wave.

It is generated as following by:

$$\text{Second phase } f(x) \cong 2^{16} - f(\pi 2^{16} - x) \{ \frac{\pi}{2} 2^{16} \leq x \leq \pi 2^{16}, x: int \} \{0 \leq f(x) \leq 2^{16}\}$$

$$\text{Third phase } f(x) \cong 2^{16} - f(x - \pi 2^{16}) \{ \pi 2^{16} \leq x \leq \frac{3\pi}{2} 2^{16}, x: int \} \{0 \leq f(x) \leq 2^{16}\}$$

$$\text{Forth phase } f(x) \cong f(2\pi 2^{16} - x) \{ \frac{3\pi}{2} 2^{16} \leq x \leq \pi 2^{16}, x: int \} \{0 \leq f(x) \leq 2^{16}\}$$

Illustration: <https://www.desmos.com/calculator/knk3zofp42>

Controlling and configuring the pulse width modulation block

The modulation block has PWM_COUNT output signals, each one is independent from the other (PWM_COUNT default =3). however, they are controlled (enabled and configured) using the same register 'pwm_en_config_reg'

Configuring the Modulation block

- Port i is disabled by writing 0 and enabled by writing 1 on bit [4i] of the 'pwm_en_config_reg' register. Disabling a port results an output of 0. Enabling the port results in generating a PWM/SPWM.
- Port i polarity is controlled by bit [4i +1]. Setting this bit to 0 means that the ON-period will be represented by logic LOW (0), and the OFF-period will be represented by logic HIGH (1). While setting them to 1 will represent the ON-period with logic HIGH (1) and the OFF-period with logic LOW (0).
- Port i modulation type is controlled by bit [4i +2]. Setting this bit to 0 generates an PWM signal. While setting it to 1 will generate an SPWM signal.
- Port i SPWM phase shift activation is controlled by bit [4i +3]. When the modulation type is PWM this bit is inactive. Setting this bit to 0 will disable the SPWM phase shift while setting them to 1 will enable it.

Adjusting the frequency of the PWM signal

For each port, two parameters are responsible for adjusting the frequency of the port's PWM signal. The frequency resolution is set in a separate register for each port `pwm_freq_rsl_config_reg[i]`, while the pwm prescaler parameters for the ports are in one register `pwm_prescaler_config_reg`. Port `i` `pwm_prescaler` is on `pwm_prescaler_config_reg[4i:4i+3]`. The `pwm_prescaler` value for each port should be in the 0 – 10 range. The frequency resolution register value is mapped to (1-256).

Adjusting the duty cycle of the PWM signal

The duty cycle of for port `i` can be adjusted using `pwm_duty_cycle_config_reg[i]`, it represents the percentage that the PWM signal will stay High within one period. This value is represented as shown:

$$\text{duty cycle}_{\text{port}[i]} = \frac{T_{on}}{T_{period}} = \frac{\text{pwm_duty_cycle_config_reg}[i]}{256}$$

Adjusting the frequency of the SPWM signal

SPWM signal is generated by comparing a sinusoidal signal with a triangle signal. The frequency of these two signals can be adjusted using the following general equation:

$$\text{signal}_{\text{freq}} = \text{Default}_{\text{freq}} \frac{\text{Multiplier}}{\text{Divider}}$$

where the default frequency is the following equation:

$$\text{Default}_{\text{sinusoidal freq}} = \frac{\text{clock}_{\text{freq}}}{2\pi \cdot 2^{16}}$$

$$\text{Default}_{\text{triangle freq}} = 2 \frac{\text{clock}_{\text{freq}}}{2\pi \cdot 2^{16}}$$

Each signal has two parameters that control the frequency, the numerator and the denominator. Each parameter is controlled by a specific register. For each port, 4 registers control the sinusoidal and the triangle signals frequency as follows:

Sinusoidal signal:

- `spwm_m_step_config_reg[i]`
- `spwm_d_step_config_reg[i]`

Triangle signal:

- `trng_m_step_config_reg[i]`
- `trng_d_step_config_reg[i]`

Therefore, port n equations can be represented by:

$$\text{Sinusoidal signal}_{\text{freq}} = \text{Default}_{\text{sinusoidal freq}} \frac{\text{spwm_m_step_config_reg}[i]}{\text{spwm_d_step_config_reg}[i]}$$

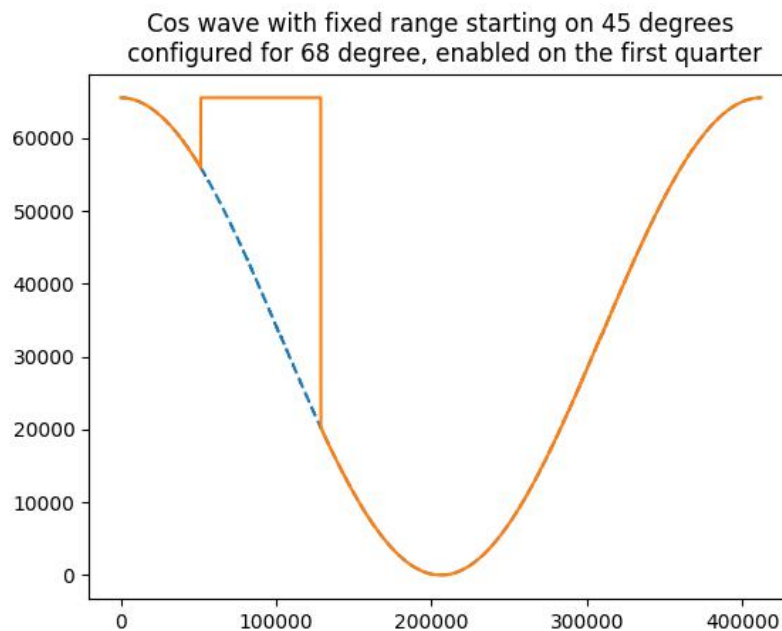
$$\text{Triangle signal}_{\text{freq}} = \text{Default}_{\text{triangle freq}} \frac{\text{trng_m_step_config_reg}[i]}{\text{trng_d_step_config_reg}[i]}$$

Note that theses registers values are mapped from (0-255) to (1-256).

Configuring the Fixed Range

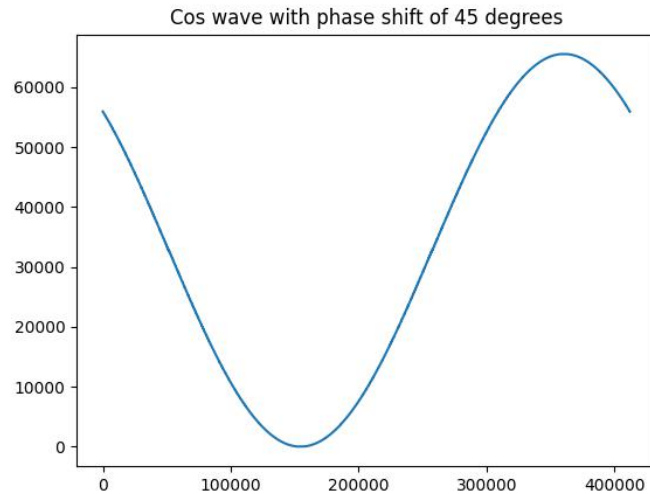
The SPWM can be fixed for a range (Fixed logic 0 or Fixed logic 1 according to polarity). For each port [i], the `spwm_fixed_range_qtr_en_config_reg[4i+3:4i]` controls the enable for the port, each bit is responsible for enabling the fixed range feature for the corresponding quarter.

For each port i, a starting angle and range can be configured in the `spwm_fixed_range_starting_angle_config_reg[i]` and `spwm_fixed_range_config_reg[i]` respectively. The value of the `spwm_fixed_range_starting_angle_config_reg[i]` will specify the starting angle (range 0-180, mapped to 0-90°) in the quarter that the fixed range will be applied at, while the `spwm_fixed_range_config_reg[i]` value will specify the width that the fixed range (range 0-180, mapped to 0-90°).



Configuring the Phase shift

When SPWM phase shift is enabled for port i, it can be configured using phase shift register. The `spwm_phase_shift_config_reg[i]` controls the phase shift (range 0 - 2880 mapped to 0-360°). The phase shift represents the sin shifted to the left by the phase shift value.



MULTI LEVEL ARBITRATION BLOCK

Multi level arbitration design includes three arbitration levels: The **weighted round robin arbitration**, the **random arbitration**, and the **weighted random arbitration**. The following figure shows the block diagram and the input/output signals of the full design.

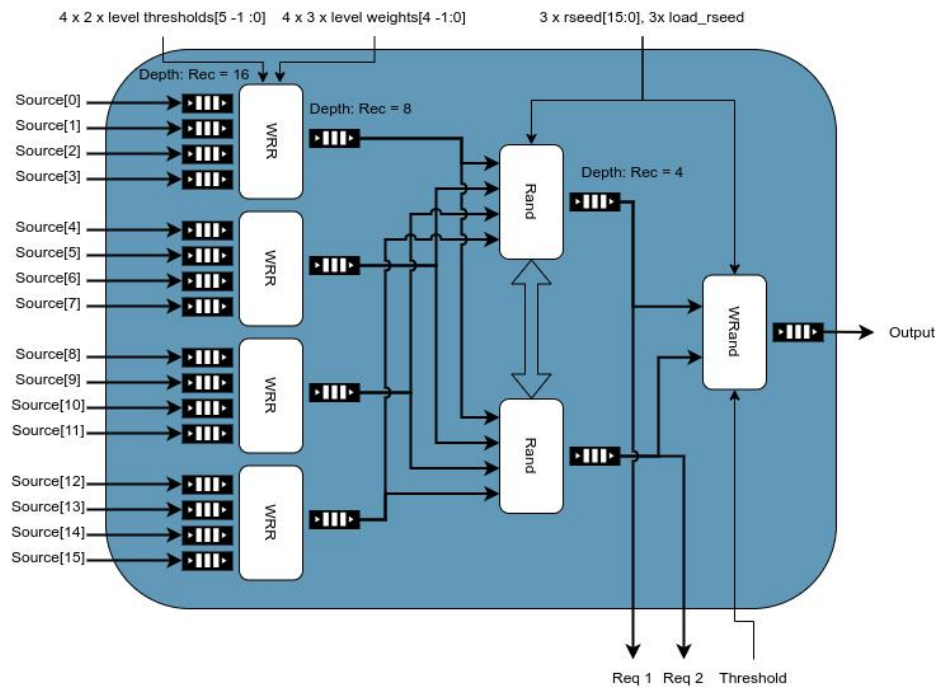


Figure 4: block diagram of the Blue-Sand design

The illustration in Fig.1 shows the internal units (blocks) of the design. The design takes input from 16 sources numbered 0 to 15, each source pushes requests to a queue with depth of 16 requests. Each weighted round robin arbitration block takes input from 4 queue sources, level thresholds configuration, and level weights configuration, then chooses a source request from the inputs and store it in the WRR queue with depth of 8 requests. The 2 random arbitration blocks take inputs from the 4 WRR queues then each choose one request from the queues without choosing the same requests, then each block stores the chosen requests in a queue with depth of 4 requests . The weighted random block takes inputs from the 2 Rand queues then chooses one of them and then put as an output. Each random and weighted random take a seed for randomization.

The design consists of 4 blocks types as described below:

- The queue block, which pushes and pulls requests.

- The weighted round robin block, takes 4 inputs and chooses one of them and put it as an output.
- The random block, takes 4 inputs and chooses one of them and put it as an output.
- The weighted random block, takes 2 inputs and chooses one of them and put it as an output.

Weighted Round Robin block

The weighted round robin block uses weighted round robin arbitration scheme to choose a request from 4 sources, with weights representing the current state of the input queues. It uses the occupancy of each source to calculate the weight/priority using level thresholds and level weights configurations. The level thresholds are used to determine the current region of the source and the level weights are used to determine the weight of each region.

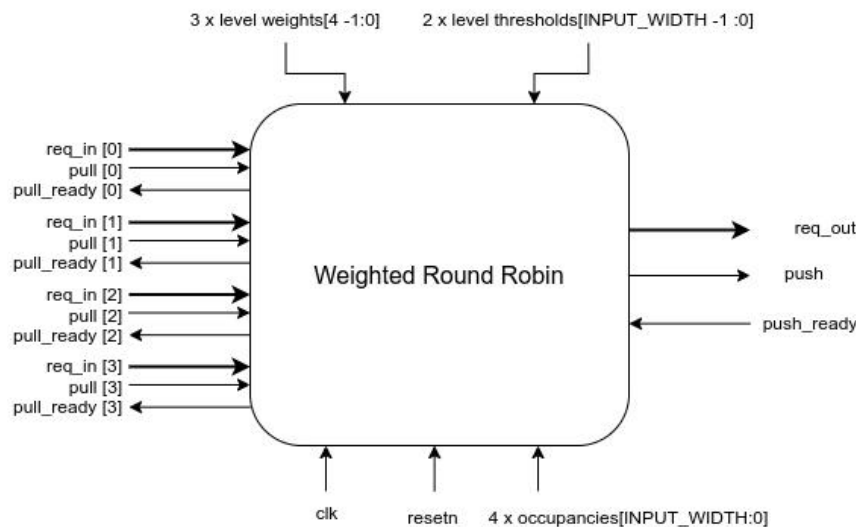


Figure 5: block diagram of the weighted round robin design

Regions

The regions are split into three regions using the level thresholds, the current weight of the queue can be in 4 states as the following figure shows.

Weight possible states.	
0	Occupancy = 0
level_weights[0]	no condition above applies and Occupancy < level_thresholds[0]
level_weights[1]	no condition above applies and Occupancy < level_thresholds[1]
level_weights[2]	no condition above applies

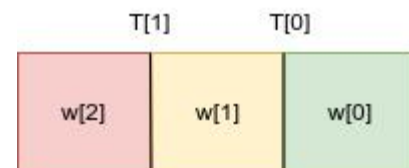


Figure 6: Weights distribution

Table 2 Weights values

Choosing a source

The WRR block uses pocket counters to determine the current source to choose. The WRR chooses the source with the highest counter value then the lower index. If a source doesn't have a ready request then the counter source is not chosen and the counter will be reset. When a source is chosen the value of it's counter is decreased by one. If all the counters are zero a new round begins by loading the current weight of the queues to the counter.

Random block

The random block uses random arbitration scheme to choose a request from 4 sources, using lfsr to generate the random number to choose from available inputs with equal probabilities according to the available requests. Random block takes an input called not_chosen which takes an input and asserts that the chosen value doesn't equal the not_chosen by adding 1 if equals. Random block outputs chosen_i which represents the value chosen by the random block.

The not_chosen and chosen_i are used to assure that the chosen values in the second level of the arbitration block aren't equal, thus if lfsr resulted choosing the same value as not_chosen, the chosen_i will change to chosen_i+1.

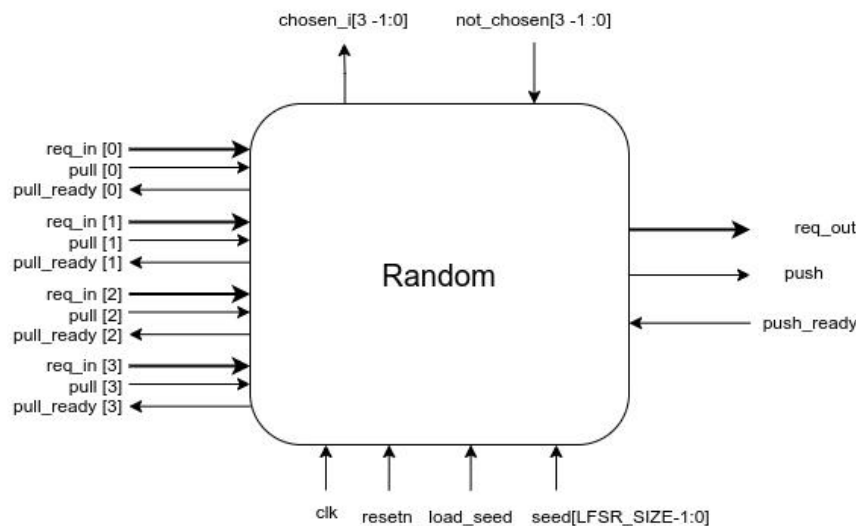


Figure 7: block diagram of the random design

Weighted Random block

The weight random block uses a weighted random arbitration scheme to choose a request from 2 sources, using lfsr to generate the random number to choose from available inputs with probability according to the threshold_val. If there is 2 available inputs the first value is chosen if and only if the random output of the lfsr is less than the threshold_val.

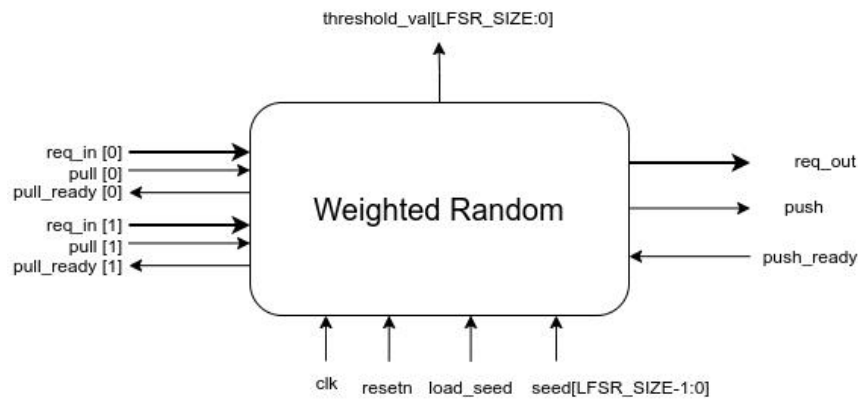


Figure 8: block diagram of the weighted random design

CLOCK MUXING UNIT

Clock muxing unit controls the operating frequency of each functional block in the system.

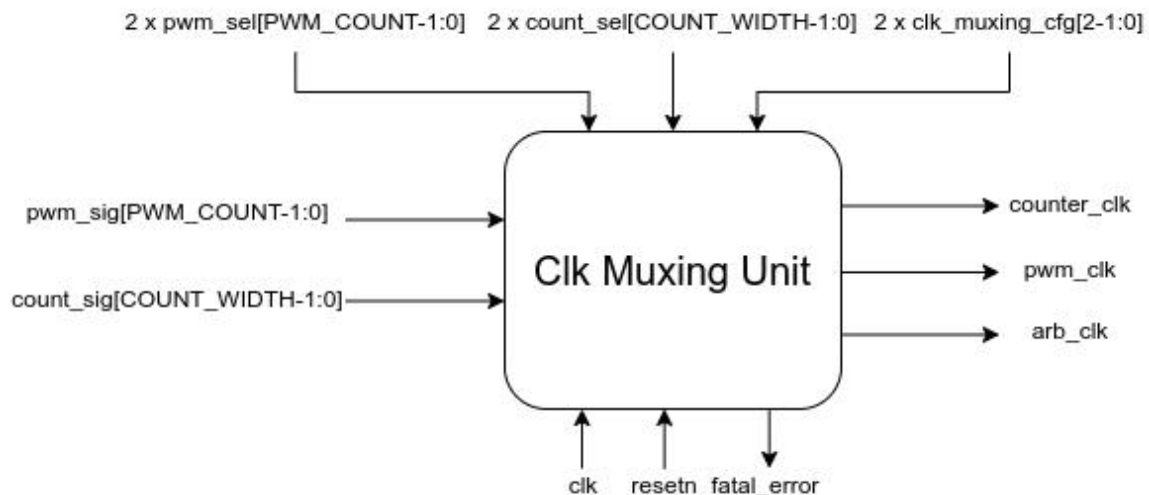


Figure 9 block diagram of clock muxing unit

Clock muxing configuration

The configuration `clk_muxing_cfg`, `count_sel`, and `pwm_sel` has two values for each, the first value with the index 0 is to configure the pwm - counter clock muxing, and the second value with index 1 is to configure the arbitration clock muxing. The following table explains the configuration.

Value of <code>clk_muxing_cfg</code>	Behaviour of pwm unit - counter unit	Behaviour of multi arbitration unit.
0	System clock is controls both.	System clock connection.
1	pwm unit will be controlled by <code>count_val</code> masked by <code>count_sel[0]</code>	arbitration unit will be controlled by <code>count_val</code> masked by <code>count_sel[1]</code>
2	counter unit will be controlled by <code>pwm_val</code> masked by <code>pwm_sel[0]</code>	arbitration unit will be controlled by <code>pwm_val</code> masked by <code>pwm_sel[0]</code>
3	Don't Care	Don't Care

Table 4 Clock muxing config

Fatal error

fatal error happens in response of wrong configuration that may cause the blue sand to enter an unknown state. Fatal error can be resolved by resetting the system and returning it to the default configuration. Fatal error is triggered when an out of range error happens causing an unknown state for the clock of a unit thus the block behaviour becomes unknown.

Signals description

This chapter details all the I/O signals in the Blue Sand design.

General signals:

1. `pclk`: oscillating between high and low state and synchronizing actions in the circuit.
2. `presetn`: which sets all the values to a predetermined state.
3. `main_interrupt`: The value is one if any unmasked interrupts are toggled.

APB Interface

APB signals can be studied from AMBA 3 APB PROTOCOL Specification.

COUNTER BLOCK INTERFACE

Table 5: counter output signals details

Port Name	Size	I/O	Description
<code>count</code>	<code>COUNT_WIDTH</code>	O	Output signal describes the count of the counter.

PULSE WIDTH MODULATION BLOCK SIGNALS

Table 6: Modulation output signals details

Port Name	Size	I/O	Description
<code>boot_done</code>	1	O	Active high signal indicates that the boot phase is done in the SPWM mode
<code>pwm_port_sig</code>	<code>PWM_COUNT</code>	O	Ports output signals that will hold the PWM/SPWM signals

MULTI LEVEL ARBITRATION BLOCK

Port Name	Size	I/O	Description
<code>req_sources</code>	16 request	I	Request input from 16 sources
<code>req_output</code>	1 request	O	Request output chosen from the arbitration
<code>pull</code>	1	I	Pull signal for the output queue, used in pull operation
<code>pull_ready</code>	1	O	Pull ready signal for the output queue, used in pull operation

push	16	I	Push signal for the 16 input queues, used in push operation
push_ready	16	O	Push ready signal for the 16 input queues, used in push operation
size_exceeded_intr	23	O	Size exceeded interrupt for all the queues in the block, with 16 queues in the input stage, 4 queues at the output of the first stage, 2 queues at the output of the second stage, and 1 output queue.
threshold_val	17	I	Threshold value for the weighted random block in the third stages.
stage3_input_requests	2 request	O	Third stage input requests which are second stage outputs.

Table 8 Multi level arbitration block signals

Weighted round robin block

Port Name	Size	I/O	Description
req_in	1 request	I	Request input from the queue.
req_out	1 request	O	Request output chosen from the arbitration.
push	1	I	Push signal used in push operation for output.
push_ready	1	O	Push ready signal used in push operation for output.
pull	4	I	Pull signal used in pull operation for inputs.
pull_ready	4	O	Pull ready signal used in pull operation for inputs.

Table 9 Weighted round robin block

Rand arbitration block

Port Name	Size	I/O	Description
req_in	1 request	I	Request input from the queue.
req_out	1 request	O	Request output chosen from the arbitration.
rseed	LFSR_SIZE	I	Seed for the lfsr.
load_rseed	1	I	Load seed for the lfsr.
push	1	I	Push signal used in push operation for output.
push_ready	1	O	Push ready signal used in push operation for output.
pull	4	I	Pull signal used in pull operation for inputs.
pull_ready	4	O	Pull ready signal used in pull operation for inputs.
chosen_i	3	O	chosen index from inputs with available requests.
not_chosen	3	I	the value that the block aren't allowed to choose.

Table 10 Rand arbitration block

Weighted random block

Port Name	Size	I/O	Description
req_in	1 request	I	Request input from the queue.
req_out	1 request	O	Request output chosen from the arbitration.
rseed	LFSR_SIZE	I	Seed for the lfsr.

load_rseed	1	I	Load seed for the lfsr.
push	1	I	Push signal used in push operation for output.
push_ready	1	O	Push ready signal used in push operation for output.
pull	2	I	Pull signal used in pull operation for inputs.
pull_ready	2	O	Pull ready signal used in pull operation for inputs.
threshold_val	LFSR_SIZE	I	Used in choosing the output request.

Table 11 Weighted random block